Übungsblatt 1: Speicherphysik und Speicher-Mapping

Mikrocomputertechnik 3 (MCT3)

Musterlösung

Zielsetzung

Sie vertiefen die Speicherphysik aus Vorlesung 1 (Von-Neumann/Harvard, SRAM/DRAM, Flash/XIP) und üben den Brückenschlag zur Embedded-C-Praxis (Linker-Sections, Pointer auf die Memory Map).

Bearbeitungszeit: ca. 45–60 Min. | **Orientierungspunkte:** Aufgabe 1: 12 P, Aufgabe 2: 10 P (Gesamt: 22 P)

Teil 1: Theorie — Speicherphysik

Aufgabe 1.1: Systemarchitekturen und Flaschenhälse

Die Architektur eines Systems bestimmt maßgeblich seine Leistung.

1. Erläutern Sie den fundamentalen Unterschied zwischen der Von-Neumann- und der Harvard-Architektur hinsichtlich ihrer Bus-Systeme.
2. Was versteht man unter dem Begriff „Von-Neumann-Flaschenhals“, und warum tritt er auf?

Musterlösung Aufgabe 1.1

1. Die **Von-Neumann-Architektur** nutzt einen gemeinsamen Speicher und einen gemeinsamen Bus für Befehle (Code) und Daten. Die **Harvard-Architektur** trennt diese Speicherräume und verwendet getrennte Busse zur CPU (Instruction- und Data-Pfad).
2. Der **Von-Neumann-Flaschenhals** entsteht, weil Fetch (Befehl laden) und Datenzugriff denselben Bus teilen und daher nicht gleichzeitig stattfinden können. Das führt zu Verzögerungen (Stalls) in der Pipeline.

Aufgabe 1.2: Volatiler Arbeitsspeicher (SRAM vs. DRAM)

Arbeitsspeicher (RAM) wird primär in SRAM und DRAM unterteilt.

1. Vergleichen Sie den physikalischen Aufbau (Anzahl und Art der Bauteile) einer SRAM-Zelle mit einer DRAM-Zelle.
2. Warum muss DRAM periodisch aufgefrischt werden (Refresh), während SRAM seinen Zustand ohne Auffrischen hält?
3. Wo werden diese beiden Speichertypen innerhalb der Speicherhierarchie eines System-on-Chip (SoC) typischerweise eingesetzt — und warum?

Musterlösung Aufgabe 1.2

1. Eine klassische **SRAM-Zelle** besteht aus sechs Transistoren (6T: zwei kreuzgekoppelte Inverter plus zwei Zugriffstransistoren). Eine **DRAM-Zelle** ist auf ein Minimum reduziert: ein Transistor und ein Kondensator (1T1C).
2. DRAM speichert die Information als Ladung im Kondensator. Die Isolation ist nicht perfekt — die Ladung leckt ab und muss periodisch (z. B. alle 64 ms) aufgefrischt werden. SRAM hält den Zustand durch das Gleichgewicht der Inverter, solange die Versorgungsspannung anliegt.
3. **SRAM** ist schnell, aber flächenintensiv/teuer → typisch CPU-nah (Register, L1/L2/L3-Caches, On-Chip-SRAM). **DRAM** ist dichter und kostengünstiger, aber langsamer → typisch Hauptspeicher (Main Memory).

Aufgabe 1.3: Non-Volatile Memory (Flash)

Für Firmware in Embedded Systems wird häufig Flash-Speicher verwendet.

1. Erklären Sie das Prinzip von Execute-In-Place (XIP) in Verbindung mit NOR-Flash.
2. Warum ist NAND-Flash trotz höherer Speicherdichte typischerweise nicht direkt für XIP durch die CPU geeignet?
3. Was versteht man unter Wear Leveling bei Flash-Speichern, und warum ist diese Technik notwendig?

Musterlösung Aufgabe 1.3

1. **XIP** bedeutet: Die Fetch-Stufe der CPU liest Instruktionen direkt über den Bus aus dem NOR-Flash und führt sie aus — ohne den Code vorher vollständig in den Arbeitsspeicher kopieren zu müssen. NOR-Flash erlaubt dafür wahlfreien Lesezugriff auf Wortebene.
2. **NAND-Flash** ist auf seiten-/blockweises Lesen und Schreiben ausgelegt (Pages, oft KiB-Größe). Es bietet keinen effizienten wahlfreien Wortzugriff für den Instruction Fetch. Deshalb ist NAND nicht direkt XIP-tauglich; Code wird typisch erst in RAM geladen oder über zusätzliche Controller/Caches bereitgestellt.
3. Flash-Zellen verschleißen bei Schreib-/Löschzyklen (hohe Spannungen belasten die Oxidisolation des Floating Gates). **Wear Leveling** (oft im Flash Translation Layer) verteilt Schreibzugriffe möglichst gleichmäßig über physische Blöcke, damit einzelne Sektoren nicht vorzeitig ausfallen.

Teil 2: Praxis — Speicher-Mapping und C-Pointer

Aufgabe 2.1: Memory Map und Linker-Skript

In einem Embedded System müssen Software und Hardware über das Speicher-Mapping verbunden werden.

1. Erklären Sie die Aufgabe eines Linker-Skripts (`.ld`-Datei) im Kontext eines Mikrocontrollers.
2. In welche physikalischen Speicherbereiche mappt der Linker typischerweise die Sections `.text` und `.data`?

Musterlösung Aufgabe 2.1

1. Das **Linker-Skript** beschreibt die Memory Map des SoCs für den Linker: Welche Software-Sections (`.text`, `.data`, `.bss`, …) liegen in welchen Speicherregionen (Flash, SRAM, …) und an welchen Adressen. Ohne diese Zuordnung wüsste die Toolchain nicht, wohin die Firmware platziert werden muss.
2. `.text` (Maschinencode, oft auch `const`-Daten) → typisch **nichtflüchtiger Flash** (z. B. NOR, XIP). `.data` (initialisierte Variablen) → zur **Laufzeit im SRAM**; die Initialwerte liegen oft zusätzlich als Ladeadresse (LMA) im Flash und werden beim Start nach SRAM kopiert. `.bss` (uninitialisierte/nullgesetzte Variablen) → ebenfalls **SRAM**.

Aufgabe 2.2: Pointer auf die Memory Map

Ein Mikrocontroller hat seinen NOR-Flash an der Basisadresse `0x08000000` gemappt.

1. Schreiben Sie ein kurzes C-Snippet, das einen Pointer auf diese Adresse definiert und den 32-Bit-Wert dort ausliest.
2. Erklären Sie, warum ein Schreibversuch über einen einfachen Store (z. B. `*ptr = 42;`) auf diese Adresse typischerweise zu einem Hardware-Fehler führt.

Musterlösung Aufgabe 2.2

```c
#include <stdint.h>

/* 1. Pointer auf die Basisadresse des NOR-Flash */
const uint32_t *firmware_word =
    (const uint32_t *)0x08000000u;

/* 2. 32-Bit-Wert lesen (wahlfreier Zugriff / XIP-Pfad) */
uint32_t ver = *firmware_word;
```

Ein direkter Schreibversuch (z. B. nach Cast auf nicht-`const`) scheitert typischerweise an der **Flash-Hardware**: Zellen lassen sich nicht wie SRAM byteweise per Store beschreiben. Zuerst sind blockweise Löschzyklen (Erase) mit speziellen Programmiersequenzen nötig. Die Speicher-Hardware blockiert den normalen CPU-Store — je nach Architektur als Bus-/Access-Fault bzw. Hard Fault.

Hinweis: Bereits `const` verhindert den Schreibversuch zur Compile-Zeit; der Hard-Fault-Aspekt betrifft den Fall eines erzwungenen Stores auf die Flash-Region.

Aufgabe 2.3: Adressraum und Endadresse

Ein Mikrocontroller mappt 1 MiB NOR-Flash ab der Basisadresse `0x08000000`.

1. Berechnen Sie die erste Adresse *nach* dem Flash-Bereich (Exclusive-Endadresse). Geben Sie den Rechenweg an.
2. Liegt die Adresse `0x080FFFFF` noch im Flash? Begründen Sie kurz.

Musterlösung Aufgabe 2.3

1. $1\,\text{MiB} = 2^{20}\,\text{Byte} = 1\,048\,576\,\text{Byte} = \texttt{0x100000}$ Byte.

$$\text{Endadresse}_{\text{exklusiv}} = \texttt{0x08000000} + \texttt{0x100000} = \texttt{0x08100000}$$

Gültiger Adressbereich: `0x08000000` … `0x080FFFFF` (inklusive).

2. Ja — `0x080FFFFF` ist die letzte gültige Byte-Adresse im Flash ($1\,\text{MiB} - 1$ Offset).